

REPORTES DE LABORATORIO: ESTRUCTURAS DE DATOS

SEMANA 13: ÁRBOLES BINARIOS DE BÚSQUEDA (BST)

Institución: Unidad Académica de Economía

Carrera: Sistemas Computacionales

Materia: Estructura de Datos Básica

Profesor: Eligardo Cruz Sánchez

Alumno: Gil Alexander Ramirez

Checkpoint de Comprensión

C1. Diagnóstico de Balanceo en el Árbol de Ejemplo

Revisando el árbol planteado (raíz 50, hijos 30 y 70, y nietos 20 y 40 bajo el 30): Sí, podemos decir que está balanceado decentemente. Si mides la altura del subárbol izquierdo nos da 1, mientras que el derecho se queda en 0. Como la diferencia entre sus alturas es de apenas 1, cumple con la regla básica para no considerarse desbalanceado.

C2. Por qué el recorrido In-Order siempre nos da un orden ascendente

Efectivamente, el recorrido in-order saca los números ordenados de menor a mayor de forma exacta. Esto pasa por la regla de oro (invariante) de los BST: cualquier nodo a la izquierda siempre va a ser más chico que la raíz, y cualquier nodo a la derecha va a ser más grande. Al aplicar el recorrido in-order, la lógica te obliga a visitar primero todo el bloque izquierdo (los menores), luego procesas el nodo actual (la raíz) y al final te vas al bloque derecho (los mayores). Por pura estructura, los datos van saliendo ordenados solitos.

C3. El problema de los datos ordenados: Árboles Sesgados

Si insertamos los datos ya ordenados (10, 20, 30, 40, 50), el árbol se deforma totalmente estirándose hacia la derecha. Queda idéntico a una lista ligada secuencial.

En este peor escenario, si queremos buscar el número 50, la complejidad se nos va a $O(n)$ porque el algoritmo no puede hacer divisiones binarias y tiene que recorrer los nodos uno por uno.

A este problema se le llama 'caso degenerado' o 'árbol degenerado', y ocurre porque el BST pierde su forma ramificada y se vuelve una línea continua.

C4. Optimización en Búsquedas por Rango (¿Por qué no es $O(n)$?)

No cae en $O(n)$ porque una buena implementación aprovecha el orden del árbol para ignorar o 'podar' caminos enteros que sabe que no sirven. Por ejemplo, si andamos buscando películas con puntajes entre 35 y 60:

- Si el código pasa por un nodo que vale 20, ya sabemos de antemano que todo su lado izquierdo tiene valores menores a 20. Así que, en lugar de perder el tiempo ahí, descartamos esa rama completa de un jalón.
- Si pasa por un nodo que vale 80, sabemos que su lado derecho tendrá puros valores mayores a 80, por lo que también lo ignoramos.

De esta forma, el algoritmo solo camina por los nodos estrictamente necesarios para llegar al rango y procesar los valores reales, ahorrando muchísimo trabajo.

Análisis Teórico y Desarrollo Técnico

Reflexión del Trabajo con el Algoritmo:

El método que de verdad nos hizo sudar fue el de 'buscar_rango'. Al principio nuestro chip mental estaba acostumbrado a revisar todo el árbol de arriba a abajo de forma mecánica. Nos costó entender que la magia del BST consiste en saber cuándo detenerse y qué ramas ignorar en la recursión. Con la guía de la materia entendimos que el reto era afinar las condiciones de decisión: cuándo movernos solo a la izquierda, cuándo solo a la derecha, y cuándo valía la pena abrir la búsqueda hacia ambos lados. Si no haces estas podas lógicas, en un árbol con millones de registros el rendimiento se te desploma por completo y terminas visitando cosas que por pura matemática nunca iban a entrar en el rango.

D1. Detección de Fallas en la Invariante del BST:

Analizando el árbol defectuoso del ejercicio (raíz 50, con un nodo 60 metido debajo del 30 en el lado izquierdo):

El culpable es claramente el nodo 60. Como está ubicado en la rama izquierda de la raíz principal (50), rompe la regla fundamental de que todo lo que esté a la izquierda de un nodo debe ser estrictamente menor que él. Como 60 es mayor que 50, la estructura deja de ser un BST válido y arruina los algoritmos de búsqueda.

D3. Comportamiento del Speedup (BST vs Diccionario):

El speedup (la diferencia de velocidad) cambia mucho según el tamaño de los datos (n) y lo que le pidas hacer. Si el árbol se mantiene balanceado, su altura crece de forma logarítmica, lo que hace que las búsquedas sigan siendo bastante rápidas a gran escala. Además, para recorrer los datos ordenados o filtrar por rangos, el BST es buenísimo porque aprovecha su estructura para podar ramas.

Sin embargo, el diccionario juega en otra liga para búsquedas exactas directas, ya que su acceso por tabla hash es $O(1)$ con constantes muy pequeñas. Por eso, el dict gana de calle en búsquedas simples de un ID, pero el BST toma la delantera cuando necesitamos reportes ordenados o filtros entre ciertos límites.

Bitácora de Validación y Decisiones del Proyecto

Resultados de las Pruebas (VALIDA):

- T1, T3 y T5 pasaron sin problemas en el primer intento.
- T2 nos rebotó porque metimos operadores de comparación estricta ($<$ y $>$) y nos estábamos comiendo los valores de los extremos del rango. Tuvimos que cambiarlo para que incluyera las igualdades.
- T4 también falló al principio porque el código no aplicaba la poda y revisaba ramas de más, haciendo que el algoritmo funcionara pero de forma muy lenta. Lo arreglamos metiendo los condicionales de descarte.

Un caso crítico de ejemplo: si tuviéramos 1,000,000 de registros balanceados y buscamos valores en un rango pequeñito (como del 500,000 al 50,010), un código sin poda haría un desastre de $O(n)$ revisando casi todo el árbol para darnos solo 10 resultados. Con la poda correcta, el costo se reduce a $O(\log n + k)$, manteniéndose súper eficiente.

Estrategia elegida para el Hito 3 de StreamMX (IA-REFLEXION-P):

Para el proyecto de StreamMX, el equipo decidió que la mejor estrategia para mitigar el riesgo de que el árbol se vuelva lento es mezclar o aleatorizar un poco los datos antes de insertarlos en el BST. Hacemos esto porque en la plataforma los títulos nuevos suelen entrar ya acomodados por fecha o puntaje, lo que provocaría que nuestro árbol básico se deforme por completo (el peor caso de $O(n)$).

Elegimos esta solución porque es sumamente fácil de programar y nos evita la complejidad de implementar algoritmos pesados de auto-balanceo como los árboles AVL o Red-Black a estas alturas de la materia. También pensamos en hacer reconstrucciones periódicas del árbol, pero vimos que gastaría más procesamiento. Aleatorizar la carga de datos es una salida práctica, limpia y suficiente para lo que el proyecto ocupa ahorita.